NLP Text Representation

Structured Notes

Text Preprocessing, Vocabulary,

One-Hot Encoding,
Word Embeddings, Word2Vec, and Co-Occurrence Matrices

Contents

1 Why NLP is Different from Numerical Data **2**
1.1 Why NLP is Challenging . . . 2
1.1.1 Sequential Nature . . . 2
1.1.2 Vocabulary Challenge . . . 2
1.1.3 High Dimensional Problem . . . 2

2 Text Preprocessing Pipeline **2**
2.1 Why Preprocessing is Required . . . 3
2.2 The Full Pipeline . . . 3
2.2.1 Step 1 — Tokenization . . . 3
2.2.2 Step 2 — Lower Casing . . . 3
2.2.3 Step 3 — Stop Words Removal . . . 3
2.2.4 Step 4 — Stemming . . . 3
2.2.5 Step 5 — Lemmatization . . . 4
2.3 Complete Pipeline Example . . . 4
2.4 Implementation with NLTK . . . 4

3 Vocabulary Creation and Word Indexing **5**
3.1 Building the Vocabulary . . . 5
3.2 Converting Sentences to Numbers . . . 5
3.3 Implementation . . . 5
3.4 The Problem with Numeric Indexing . . . 6

4 One-Hot Encoding **6**
4.1 One-Hot Vectors . . . 6
4.2 Encoding a Sentence . . . 7
4.3 Implementation . . . 7
4.4 Advantages of One-Hot Encoding . . . 7
4.5 Limitations of One-Hot Encoding . . . 7

5 Word Embeddings **7**
5.1 From Sparse to Dense Vectors . . . 8
5.2 Semantic Meaning in Embeddings . . . 8
5.3 Vector Arithmetic with Embeddings . . . 8
5.4 Why Embeddings Work: The Distributional Hypothesis . . . 8
5.5 Implementation . . . 8

6 Word2Vec **9**
6.1 Context Window . . . 9
6.2 Two Architectures . . . 9
6.2.1 Skip-Gram . . . 9
6.2.2 CBOW (Continuous Bag of Words) . . . 10
6.3 Why Word2Vec Learns Semantic Meaning . . . 10
6.4 Implementation: Generating Training Pairs . . . 10

7 Co-Occurrence Matrix **10**
7.1 Building the Matrix . . . 10
7.2 Interpretation . . . 11
7.3 Implementation . . . 11
7.4 Word2Vec vs Co-Occurrence Matrix . . . 12

8 Summary **12**

1 Why NLP is Different from Numerical Data

The Text Data Challenge

Unlike images (pixel values), tabular data (numbers), or time series (continuous signals), **text data is inherently non-numerical**. Neural networks require numerical input, so text must be converted into numbers before processing.

Consider these example sentences:

- "I love machine learning"
- "Deep learning is powerful"
- "AI is transforming industries"

None of these are numerical values. An RNN expects input of the form $X = [x_1, x_2, x_3, \ldots, x_T]$, but raw text like "I love NLP" cannot be fed directly.

1.1 Why NLP is Challenging

1.1.1 Sequential Nature

Word order matters in language. Consider:

- "I love NLP" $\neq$ "NLP love I"

Language is **sequence dependent** — the meaning changes when the order changes.

1.1.2 Vocabulary Challenge

Given a small dataset:

- "I love NLP"
- "I love Deep Learning"
- "Deep learning is powerful"

The vocabulary becomes: $V = \{\text{I, love, NLP, Deep, Learning, is, powerful}\}$. As more text is added, vocabulary size grows rapidly.

1.1.3 High Dimensional Problem

If vocabulary size = 10,000, each word must be represented in a 10,000-dimensional space. This creates:

- Memory issues
- Computation issues
- Learning difficulty

Bridge to RNN

RNNs require numeric sequences. The conversion pipeline is: **Text** $\rightarrow$ **Numbers** $\rightarrow$ **RNN**. This entire section covers the "Text $\rightarrow$ Numbers" step.

2 Text Preprocessing Pipeline

Text Preprocessing

Text preprocessing is the process of cleaning and standardizing raw text before converting it into numerical representations. It reduces vocabulary size, improves model learning, reduces computation, and improves performance.

2.1 Why Preprocessing is Required

Consider these raw sentences:

- "I love Machine Learning!!!"
- "I LOVE machine learning"
- "I love ML"

For humans, these have the same meaning. For machines, they are completely different strings. Without preprocessing, this creates vocabulary explosion, poor learning, and data inconsistency.

2.2 The Full Pipeline

> **Raw Text → Tokenization → Lowercase → Stopword Removal → Stemming / Lemmatization**

2.2.1 Step 1 — Tokenization

Tokenization splits a sentence into individual words (tokens).

> **Input:** "I love NLP"
>
> **Output:** ["I", "love", "NLP"]
>
> Mathematically, a sentence $S = [w_1, w_2, w_3, \ldots, w_n]$

2.2.2 Step 2 — Lower Casing

Converting all text to lowercase ensures "I" and "i" map to the same token.

> **Input:** "I Love NLP"
>
> **Output:** "i love nlp"

2.2.3 Step 3 — Stop Words Removal

Common words that carry little semantic meaning are removed. Examples of stop words: *is, the, and, are, of.*

> **Input:** "This is a machine learning model"
>
> **Output:** ["machine", "learning", "model"]

2.2.4 Step 4 — Stemming

Stemming reduces words to their root form by applying heuristic rules (suffix stripping).

Word	Stemmed
running	run
played	play
learning	learn
studies	studi *(incorrect)*

Limitation

Stemming can produce incorrect roots. For example, "studies" becomes "studi" instead of

"study". This is because stemming applies simple suffix-stripping rules without understanding word meaning.

2.2.5 Step 5 — Lemmatization

Lemmatization is a more accurate version of stemming that uses vocabulary and morphological analysis to return the dictionary form of a word.

Word	Lemmatized
running	run
better	good
studies	study

Lemmatization understands **word meaning** and produces valid dictionary words.

2.3 Complete Pipeline Example

Raw: "I am learning machine learning and loving it"

After preprocessing: ["learn", "machine", "learn", "love"]

2.4 Implementation with NLTK

Text Preprocessing with NLTK

```python
import nltk
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from nltk.corpus import wordnet

sentence = "I am learning machine learning and loving it"

# Lowercase
sentence = sentence.lower()

# Tokenization
tokens = nltk.word_tokenize(sentence)

# Remove punctuation
tokens = [re.sub(r'[^\w\s]', '', word) for word in tokens]
tokens = [word for word in tokens if word != ""]

# Remove stopwords
stop_words = set(stopwords.words('english'))
tokens = [word for word in tokens if word not in stop_words]

# Stemming
stemmer = PorterStemmer()
stemmed = [stemmer.stem(word) for word in tokens]

# Lemmatization with POS tagging
def get_wordnet_pos(word):
    tag = pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN,
                "V": wordnet.VERB, "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)
```

```
34
35 lemmatizer = WordNetLemmatizer()
36 lemmatized = [lemmatizer.lemmatize(word, get_wordnet_pos(word))
37               for word in tokens]
38
39 print("Tokens:", tokens)        # ['learning', 'machine', 'learning', '
       loving']
40 print("Stemmed:", stemmed)      # ['learn', 'machin', 'learn', 'love']
41 print("Lemmatized:", lemmatized) # ['learn', 'machine', 'learn', 'love']
```

Listing 1: Full preprocessing pipeline

Stemming vs Lemmatization

Stemming is faster but less accurate ("machin" from "machine"). Lemmatization is slower but produces valid dictionary words ("machine" stays "machine"). Lemmatization requires POS tagging for best results.

3 Vocabulary Creation and Word Indexing

After preprocessing, tokens (words) are obtained. However, neural networks cannot understand words — they only understand numbers.

Vocabulary

A **vocabulary** is the set of all unique words extracted from a dataset. Each word is assigned a unique integer index, creating a **word-to-index mapping** (word dictionary).

3.1 Building the Vocabulary

Given the dataset:

- "I love NLP"
- "I love Deep Learning"
- "Deep learning is powerful"

Vocabulary: $V = \{\text{i, love, nlp, deep, learning, is, powerful}\}$

Word Indexing:
$\text{i} \to 0, \quad \text{love} \to 1, \quad \text{nlp} \to 2, \quad \text{deep} \to 3$
$\text{learning} \to 4, \quad \text{is} \to 5, \quad \text{powerful} \to 6$

3.2 Converting Sentences to Numbers

"I love NLP" $\to [0, 1, 2]$

"Deep learning is powerful" $\to [3, 4, 5, 6]$

Formally: Given vocabulary $V = \{w_1, w_2, \ldots, w_n\}$ and index mapping $f(w_i) = i$, a sentence $S = [w_a, w_b, w_c]$ becomes $S = [f(w_a), f(w_b), f(w_c)]$.

3.3 Implementation

Vocabulary Creation

```python
sentences = ["i love nlp", "i love deep learning",
             "deep learning is powerful"]

# Tokenization
tokens = [sentence.split() for sentence in sentences]

# Create vocabulary
vocab = set()
for sentence in tokens:
    for word in sentence:
        vocab.add(word)

# Word-to-index dictionary
word2index = {word: i for i, word in enumerate(vocab)}
print("Vocabulary:", word2index)

# Convert sentence to numeric
example = "i love nlp".split()
numeric = [word2index[word] for word in example]
print("Numeric:", numeric)   # e.g., [5, 0, 1]
```

Listing 2: Building vocabulary and converting sentences

3.4 The Problem with Numeric Indexing

False Relationships

Simple integer indexing introduces **false numeric relationships**. For example:

- king $\rightarrow$ 1, queen $\rightarrow$ 2
- The model may interpret: queen $>$ king (because $2 > 1$)
- This is incorrect — numbers are arbitrary labels, not meaningful values

This motivates the need for **One-Hot Encoding**.

4 One-Hot Encoding

One-Hot Encoding

One-Hot Encoding represents each word as a binary vector where exactly one position is 1 and all others are 0. The vector dimension equals the vocabulary size $|V|$.

4.1 One-Hot Vectors

Given vocabulary $V = \{\text{king, queen, man, woman}\}$:

$$\text{king} \rightarrow [1, 0, 0, 0]$$
$$\text{queen} \rightarrow [0, 1, 0, 0]$$
$$\text{man} \rightarrow [0, 0, 1, 0]$$
$$\text{woman} \rightarrow [0, 0, 0, 1]$$

Properties:

- Only one position is 1; all others are 0
- Vector size = Vocabulary size $|V|$
- Each word has a unique vector

4.2 Encoding a Sentence

Vocabulary: $V = \{\text{I, love, NLP}\}$

$\text{I} \to [1,0,0], \quad \text{love} \to [0,1,0], \quad \text{NLP} \to [0,0,1]$

$$\text{“I love NLP”} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4.3 Implementation

One-Hot Encoding

```python
import numpy as np

sentences = ["i love nlp", "i love deep learning"]

# Build vocabulary
vocab = list(set(" ".join(sentences).split()))
word2index = {word: i for i, word in enumerate(vocab)}

def one_hot(word):
    vector = np.zeros(len(vocab))
    vector[word2index[word]] = 1
    return vector

print("Vocabulary:", vocab)
print("One-Hot for 'deep':", one_hot("deep"))
# Output: [0. 0. 1. 0. 0.]
```

Listing 3: Creating one-hot vectors with NumPy

4.4 Advantages of One-Hot Encoding

- No numeric bias — no false ordering between words
- Each word has a unique representation
- Eliminates the false relationship problem of integer indexing

4.5 Limitations of One-Hot Encoding

Critical Limitation: No Semantic Similarity

One-hot vectors make all words **equally distant** from each other:

- king $\to [1,0,0,0]$, queen $\to [0,1,0,0]$
- Cosine similarity between any two one-hot vectors $= 0$
- But “king” and “queen” are semantically related

Additionally, if vocabulary size $= 10{,}000$, each vector has 10,000 dimensions — extremely sparse and memory-intensive.

5 Word Embeddings

Word Embeddings

Word embeddings are dense, low-dimensional vector representations of words where semantically similar words have similar vectors. Unlike one-hot encoding (sparse, high-dimensional), embeddings are compact and capture meaning.

5.1 From Sparse to Dense Vectors

Property	One-Hot Encoding	Word Embeddings
Dimensionality	Vocabulary size (e.g., 10,000)	Chosen size (e.g., 100)
Sparsity	Sparse (mostly zeros)	Dense (all values used)
Similarity	All words equidistant	Similar words close
Memory	High	Low

5.2 Semantic Meaning in Embeddings

Dense vectors capture relationships between words:

king $\rightarrow [0.21, 0.45, 0.78, 0.12]$
queen $\rightarrow [0.23, 0.48, 0.75, 0.15]$

$These vectors are$ **similar**, $so the model understands: king \approx$ queen.

5.3 Vector Arithmetic with Embeddings

The most remarkable property of word embeddings is that they support meaningful arithmetic:

king - man + woman $\approx$ queen

Paris $-$ France $+$ Italy $\approx$ Rome

Walking $-$ Walk $+$ Run $\approx$ Running

This demonstrates that embeddings capture gender, geography, tense, and other semantic relationships **mathematically**.

5.4 Why Embeddings Work: The Distributional Hypothesis

Distributional Hypothesis

Words that appear in similar contexts have similar meanings. For example:

- "I drink **coffee**"
- "I drink **tea**"

Since "coffee" and "tea" appear in the same context ("I drink ___"), their embeddings become similar.

5.5 Implementation

Word Embedding Arithmetic

```python
import numpy as np

# Example embeddings (manually defined for intuition)
embeddings = {
```

```
    "king":  np.array([0.8, 0.2, 0.6]),
    "queen": np.array([0.82, 0.25, 0.58]),
    "man":   np.array([0.7, 0.1, 0.5]),
    "woman": np.array([0.75, 0.15, 0.55])
}

# king - man + woman
result = embeddings["king"] - embeddings["man"] + embeddings["woman"]
print("king - man + woman:", result)   # [0.85, 0.25, 0.65]
print("queen:             ", embeddings["queen"])  # [0.82, 0.25, 0.58]
# Result is close to queen -- semantic relationship captured!
```

Listing 4: Demonstrating embedding vector arithmetic

6 Word2Vec

Word2Vec

Word2Vec is an algorithm that learns word embeddings by training a shallow neural network on word co-occurrence patterns. It uses **context** to learn embeddings: words appearing in similar contexts get similar vectors.

6.1 Context Window

Given a sentence and a window size, Word2Vec creates (word, context) training pairs.

Sentence: "I love deep learning"

Window size = 1

Training pairs:
(I, love), (love, I), (love, deep), (deep, love), (deep, learning), (learning, deep)

6.2 Two Architectures

Word2Vec has two model architectures:

Aspect	Skip-Gram	CBOW
Task	Given a word, predict its context words	Given context words, predict the center word
Example Input	"love"	["I", "deep"]
Example Output	["I", "deep"]	"love"
Best for	Small datasets, rare words	Large datasets, frequent words

6.2.1 Skip-Gram

Given a center word, predict surrounding context words.

Sentence: "I love deep learning"

Center word: "love"
Predicted context: "I", "deep"

6.2.2 CBOW (Continuous Bag of Words)

Given surrounding context words, predict the center word.

> **Sentence:** "I love deep learning"
>
> **Context:** "I", "deep"
> **Predicted center word:** "love"

6.3 Why Word2Vec Learns Semantic Meaning

Words with similar contexts learn similar embeddings:

- "I drink **coffee**" — coffee appears with context (I, drink)
- "I drink **tea**" — tea appears with context (I, drink)

Since coffee and tea share the same context, their embedding vectors converge to similar values during training.

6.4 Implementation: Generating Training Pairs

Word2Vec Training Pairs

```python
sentence = "i love deep learning".split()
window_size = 1
pairs = []

for i, word in enumerate(sentence):
    for j in range(-window_size, window_size + 1):
        if j != 0 and 0 <= i + j < len(sentence):
            pairs.append((word, sentence[i + j]))

print("Training Pairs:")
for pair in pairs:
    print(pair)
# ('i', 'love'), ('love', 'i'), ('love', 'deep'),
# ('deep', 'love'), ('deep', 'learning'), ('learning', 'deep')
```

Listing 5: Generating Skip-Gram training pairs

7 Co-Occurrence Matrix

While Word2Vec learns from **local context** (nearby words), an alternative approach uses **global context** by counting how often words co-occur across the entire corpus.

Co-Occurrence Matrix

A **co-occurrence matrix** X is a square matrix where entry X_{ij} counts the number of times word i appears within a context window of word j across the entire corpus.

7.1 Building the Matrix

Given the sentences:

- "I love deep learning"
- "I love machine learning"
- "deep learning is powerful"

With window size = 1, count co-occurrence pairs across all sentences:

	I	love	deep	learning	machine	is	powerful
I	0	2	0	0	0	0	0
love	2	0	1	0	1	0	0
deep	0	1	0	2	0	0	0
learning	0	0	2	0	1	1	0
machine	0	1	0	1	0	0	0
is	0	0	0	1	0	0	1
powerful	0	0	0	0	0	1	0

Formally: X_{ij} = number of times word i appears within a context window of word j across the corpus.

7.2 Interpretation

Words that co-occur frequently have higher matrix values and are likely to be semantically related:

- "deep" and "learning" have a high co-occurrence value (2), indicating they are related
- "coffee" and "tea" would share high co-occurrence values with "drink", "cup", "morning" — making them similar

7.3 Implementation

Co-Occurrence Matrix

```python
import numpy as np
import pandas as pd

sentences = ["i love deep learning",
             "i love machine learning",
             "deep learning is powerful"]

vocab = list(set(" ".join(sentences).split()))
word2index = {word: i for i, word in enumerate(vocab)}

co_matrix = np.zeros((len(vocab), len(vocab)))
window_size = 1

for sentence in sentences:
    words = sentence.split()
    for i, word in enumerate(words):
        for j in range(-window_size, window_size + 1):
            if j != 0 and 0 <= i + j < len(words):
                neighbor = words[i + j]
                co_matrix[word2index[word]][word2index[neighbor]] += 1

co_df = pd.DataFrame(co_matrix, index=vocab, columns=vocab)
print("Co-Occurrence Matrix:\n")
print(co_df)
```

Listing 6: Building a co-occurrence matrix

7.4 Word2Vec vs Co-Occurrence Matrix

Aspect	Word2Vec	Co-Occurrence Matrix
Context	Local (window-based, per sentence)	Global (corpus-wide counts)
Learning	Neural network training	Direct counting / matrix factorization
Output	Dense embedding vectors	Sparse count matrix (can be reduced)
Foundation for	Word2Vec, FastText	GloVe

8 Summary

- **NLP Challenge:** Neural networks require numerical input, but text is inherently non-numerical. Word order matters, and vocabulary grows rapidly.
- **Text Preprocessing:** A pipeline of tokenization, lowercasing, stop word removal, stemming, and lemmatization cleans and standardizes text. Lemmatization is preferred over stemming for accuracy.
- **Vocabulary and Indexing:** Unique words are collected into a vocabulary and assigned integer indices. However, integer indexing introduces false numeric relationships.
- **One-Hot Encoding:** Represents words as binary vectors with one "1" per word. Eliminates false ordering but cannot capture semantic similarity, and creates high-dimensional sparse vectors.
- **Word Embeddings:** Dense, low-dimensional vectors that capture semantic meaning. Similar words have similar vectors. Support meaningful arithmetic (e.g., king $-$ man $+$ woman $\approx$ queen).
- **Word2Vec:** Learns embeddings using local context via two architectures — Skip-Gram (word $\rightarrow$ context) and CBOW (context $\rightarrow$ word). Words with similar contexts get similar embeddings.
- **Co-Occurrence Matrix:** Captures global word relationships by counting co-occurrences across the entire corpus. Forms the foundation for methods like GloVe.